

Subbab 3.2.4.1 — Versi Usulan dengan Penjelasan Per Grup

Penjelasan per grup (*Knowledge Base*, *Query Processing*, *Hybrid Scoring*) selaras dengan diagram hybrid retrieval.

Perancangan Komponen *Retrieval* pada Sistem

Perancangan komponen *retrieval* pada sistem dilakukan menggunakan pendekatan *hybrid retrieval* yang menggabungkan jalur *dense retrieval* dan *sparse retrieval*. Jalur *dense* digunakan untuk menangkap kemiripan semantik antara *query* dan dokumen melalui representasi *embedding*, sedangkan jalur *sparse* digunakan untuk menangkap kecocokan leksikal berbasis kata kunci menggunakan BM25. Kedua jalur tersebut kemudian digabungkan melalui proses normalisasi dan pembobotan skor untuk menghasilkan kandidat dokumen yang paling relevan.

Berdasarkan rancangan tersebut, komponen *retrieval* dibagi ke dalam tiga bagian utama, yaitu *knowledge base* sebagai tahap inisialisasi indeks dokumen, *query processing* sebagai tahap pemrosesan *query* dan perhitungan skor, serta *hybrid scoring* sebagai tahap penggabungan skor dan pemilihan kandidat. Ketiga bagian ini ditunjukkan pada diagram hybrid retrieval dengan warna berbeda, yaitu biru (*Knowledge Base*), hijau (*Query Processing*), dan kuning (*Hybrid Scoring*).

1. *Knowledge Base* — Fase Inisialisasi

Tahap *knowledge base* merupakan fase inisialisasi yang dijalankan sekali saat sistem dimulai. Pada tahap ini, data kontrol ISO berbentuk JSON dimuat dan dikonversi menjadi representasi teks terstruktur. Representasi tersebut kemudian diproses pada dua jalur secara paralel: jalur *dense* membentuk *embedding* dokumen, sedangkan jalur *sparse* membangun indeks BM25. Kedua indeks yang dihasilkan disimpan dalam memori dan digunakan ulang pada setiap pemrosesan *query*.

Proses Pembentukan *Embedding* Dokumen (Jalur Dense)

Proses pembentukan *embedding* dokumen bertujuan untuk mengubah seluruh kontrol ISO menjadi representasi vektor yang dapat digunakan untuk perhitungan kemiripan semantik. Proses ini dimulai dari konversi data JSON menjadi teks terstruktur, kemudian dilakukan *embedding* menggunakan model bahasa, dan diakhiri dengan normalisasi serta penyimpanan vektor hasil *embedding*.

- a. **Representasi Teks:** Menggunakan *function* `control_to_text(control: dict)`

-> `str` untuk menggabungkan field JSON menjadi satu teks utuh. Representasi ini menjadi masukan utama bagi proses *embedding* dan tokenisasi agar konteks dokumen dapat diproses secara konsisten.

- b. **Inisiasi Model dan Dimensi *Embedding*:** Menggunakan *function* `get_model()` -> `SentenceTransformer` untuk memuat model *paraphrase-multilingual-MiniLM-L12-v2* dengan mekanisme *lazy loading*. Selanjutnya, `get_embedding_dim()` -> `int` digunakan untuk memperoleh dimensi *embedding* sebesar 384 dimensi.
- c. **Pembentukan *Embedding* Teks:** Menggunakan *function* `encode_texts(texts: list[str])` -> `np.ndarray` untuk mengubah seluruh teks kontrol ISO menjadi vektor *embedding*. Hasilnya berupa *array* NumPy berdimensi (93, 384) dengan tipe data `float32`.
- d. **Normalisasi *Embedding*:** Menggunakan *function* `normalize(vectors: np.ndarray)` -> `np.ndarray` untuk melakukan normalisasi L2 pada vektor *embedding*. Normalisasi ini membuat perhitungan *dot product* setara dengan *cosine similarity* dan menjaga stabilitas ketika terdapat vektor bernilai nol.
- e. **Penyimpanan *Embedding* Dokumen:** *Embedding* dokumen disimpan pada atribut `HybridRetriever` secara *in-memory*. Penyimpanan dilakukan dalam bentuk *array* NumPy dan dibangun ulang setiap kali sistem dijalankan.

Proses Pembangunan Indeks BM25 (Jalur Sparse)

Proses pembangunan indeks BM25 bertujuan untuk menyusun struktur indeks leksikal yang memungkinkan pencarian dokumen berdasarkan kecocokan kata kunci. Proses ini dimulai dari tokenisasi seluruh dokumen, kemudian dilakukan perhitungan statistik seperti panjang dokumen, *document frequency*, dan *inverse document frequency*.

- a. ***Preprocessing* Teks untuk BM25:** Menggunakan *function* `tokenize(text: str)` -> `list[str]` untuk mengubah teks dokumen menjadi daftar *token*. Proses ini dilakukan dengan *lowercase* dan pengambilan karakter alfanumerik menggunakan *regex* `[A-Za-z0-9]+`.
- b. **Pembangunan Indeks BM25:** Pada saat `BM25Retriever` diinisialisasi, seluruh representasi teks kontrol ISO diproses menggunakan `tokenize()`, kemudian sistem menghitung panjang dokumen (`doc_lens`), *document frequency* (`_compute_doc_freq()`), dan *inverse document frequency* menggunakan formula Okapi BM25, yaitu $\log(1 + (N - df + 0.5) / (df + 0.5))$. Indeks yang dihasilkan disimpan dalam memori dan siap digunakan pada tahap *query processing*.

2. Query Processing — Pemrosesan Query dan Perhitungan Skor

Tahap *query processing* dijalankan setiap kali pengguna memberikan *query*. Pada tahap ini, *query* diproses secara paralel pada dua jalur: jalur *dense* mengubah *query* menjadi vektor *embedding* dan menghitung kemiripan semantik terhadap seluruh dokumen, sedangkan jalur *sparse* menokenisasi *query* dan menghitung skor BM25. Hasil dari kedua jalur adalah masing-masing 93 skor relevansi yang akan digunakan pada tahap *hybrid scoring*.

Proses Perhitungan Dense Score (Jalur Dense)

Proses perhitungan *dense score* bertujuan untuk mengukur kemiripan semantik antara *query* dan seluruh dokumen. *Query* diubah menjadi vektor *embedding* yang sama dengan dokumen, kemudian dilakukan perhitungan *dot product* untuk menghasilkan skor kemiripan semantik.

- a. **Pembentukan *Embedding Query*:** Menggunakan *function* `encode_texts([query])` untuk mengubah *query* menjadi vektor *embedding* berdimensi 384. Karena fungsi yang sama digunakan untuk dokumen dan *query*, keduanya direpresentasikan dalam ruang vektor yang sama.
- b. **Perhitungan *Dense Score*:** Menggunakan *function* `_dense_scores(query: str) -> np.ndarray` untuk menghitung *dot product* antara vektor *query* dan *embedding* seluruh dokumen yang telah disimpan pada fase inisialisasi. Karena kedua vektor telah dinormalisasi, nilai *dot product* ini setara dengan *cosine similarity*.

Proses Perhitungan Sparse Score (Jalur Sparse)

Proses perhitungan *sparse score* bertujuan untuk mengukur kecocokan leksikal antara *query* dan seluruh dokumen. *Query* diubah menjadi daftar *token* yang konsisten dengan dokumen, kemudian dilakukan perhitungan skor BM25 berdasarkan *term frequency* dan *inverse document frequency*.

- a. **Tokenisasi *Query*:** Pada jalur *sparse*, *query* diproses menggunakan `tokenize()` yang sama dengan dokumen. Konsistensi tokenisasi ini memastikan bahwa proses pencocokan *term* pada BM25 berjalan dengan aturan yang sama.
- b. **Perhitungan *Sparse Score*:** Menggunakan *function* `score_query(query: str) -> list[float]` untuk menghitung skor BM25 *query* terhadap seluruh dokumen berdasarkan kombinasi *term frequency*, IDF, dan normalisasi panjang dokumen.

3. Hybrid Scoring — Penggabungan Skor dan Pemilihan Kandidat

Tahap *hybrid scoring* menggabungkan skor dari jalur *dense* dan *sparse* menjadi satu nilai akhir yang merepresentasikan relevansi dokumen. Skor dari kedua jalur terlebih dahulu dinormalisasi agar memiliki skala yang sama, kemudian digabungkan menggunakan *weighted sum*. Berdasarkan skor hasil penggabungan, sistem mengurutkan dokumen dan memilih kandidat teratas yang digunakan sebagai konteks untuk tahap generasi.

- a. **Normalisasi Skor:** Menggunakan *function* `_minmax(scores: list[float]) -> np.ndarray` untuk menormalisasi skor *dense* dan *sparse* ke rentang 0 sampai 1. Normalisasi dilakukan secara terpisah karena kedua skor memiliki skala yang berbeda.
- b. **Fusion Score:** Menggunakan metode *weighted sum* untuk menggabungkan skor *dense* dan *sparse* yang telah dinormalisasi. Formula yang digunakan adalah $hybrid_score = w_{dense} \times dense_norm + w_{sparse} \times sparse_norm$, dengan bobot bawaan $w_{dense} = 0,6$ dan $w_{sparse} = 0,4$.
- c. **Final Ranking:** Setelah *hybrid score* diperoleh, sistem mengurutkan seluruh dokumen secara *descending* menggunakan `np.argsort` dan memilih 3 dokumen dengan skor tertinggi.
- d. **Pembentukan Hasil:** Menggunakan *function* `_build_result(idx)` untuk mengambil metadata kontrol terpilih, yaitu `control_id`, `title`, `objective`, dan `description`, serta skor *dense*, *sparse*, dan *hybrid*. Hasil ini dikirimkan sebagai konteks ke LLM pada tahap generasi.

Checklist Pemetaan Inti 4.1.3

Sumber	Poin inti	Masuk ke versi usulan
4.1.3.1 Dense	Representasi teks; inisiasi model; dimensi <i>embedding</i> ; pembentukan <i>embedding</i> ; normalisasi; penyimpanan <i>embedding</i> dokumen.	Ya, bagian <i>Knowledge Base</i> (Dense: a–e) dan <i>Query Processing</i> (Dense: a–b).

4.1.3.2 Sparse	<i>Preprocessing</i> ; tokenisasi dokumen; panjang dokumen; <i>document frequency</i> ; IDF; skor BM25.	Ya, bagian <i>Knowledge Base</i> (Sparse: a–b) dan <i>Query Processing</i> (Sparse: a–b).
4.1.3.3 Hybrid	Inisialisasi <i>HybridRetriever</i> ; <i>dense score</i> ; <i>sparse score</i> ; <i>method_order</i> ; <i>candidate merging</i> ; normalisasi; <i>fusion score</i> ; <i>final ranking</i> .	Ya, bagian <i>Hybrid Scoring</i> (a–d).